```
# INFO-6146 – In-class Activity #1 (10 Marks)
# Dataset: California Housing
# Objective: Assess understanding of model architecture, loss functions, training process and evaluatio

import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

np.random.seed(42)
tf.random.set_seed(42)

# SECTION 1 – LOAD DATA

data = fetch_california_housing()

X = data.data
y = data.target

print("Feature shape:", X.shape)
print("Target shape:", y.shape)


# SECTION 2 – TRAIN TEST SPLIT (FILL IN THE BLANKS)
# QUESTION 1 (1 mark):Choose an appropriate test_size and explain in comments why you selected it.
# ANSWER 1: 20% Test 80% Train split is standard. It is important to have enough data to train however.

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Training samples:", X_train.shape[0])
print("Test samples:", X_test.shape[0])


# SECTION 3 – BUILD MODEL (FILL IN THE BLANKS)
# QUESTION 2(1 mark): Build a regression model with One hidden layer with appropriate number of neurons
# Explain your choices in comments.
# ANSWER 2: Sequential not functional because thats what we are currently taught. Dense because all neu
# layer should be sufficient because you do not want to use too many resources, and "relu" is common.

model = Sequential([
    Dense(8, activation="relu", input_shape=(X_train.shape[1],)),
    Dense(1)
])

model.summary()

# SECTION 4 – COMPILE MODEL (FILL IN THE BLANKS)
# QUESTION 3(1 mark): Define the optimizer, loss function and metric
# Explain why you chose them in comments.
# ANSWER 3: mse is common for regression because it negates negativity in numbers through squaring.
# "adam" is common for tuning the gradient descent
# use accuracy because i want to understand the TP and TN of the model.

model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["accuracy"]
)


# SECTION 5 – TRAIN MODEL (FILL IN THE BLANKS)
# QUESTION 4(1 mark): Select the number epochs, batch_size and validation_split
```

```
# Explain why you picked those values in comments.
# ANSWER 4: 80/20 split in standard, need to make sure you have enough to train.
# 10 epochs is the umber of iterations on the training, too much can be excessive and use many resource
# 8 batch size because little will take too long but too much will take up lots of memeory.

history = model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=10,
    batch_size=8,
    verbose=0
)

print("Training complete.")


# SECTION 6 – PLOT LOSS CURVES (FILL IN THE BLANKS)
# QUESTION 5(1 mark): Plot training loss and validation loss.
# Comment on whether the model is overfitting or underfitting?
# ANSWER 5: model is underfitting as it perfomance suffers in validation.

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.title("Training vs Validation Loss")
plt.show()


# SECTION 7 – MODEL EVALUATION (FILL IN THE BLANKS)
# QUESTION 6(2 marks): Evaluate the model on test data and explain what the results indicate.
# ANSWER 6: The model failed testing, its loss the slope of the gradient was 0.4 very slanted and its a
# was 0.002 or 0.2%, very bad. Probably needs more epochs for training.

test_loss, test_metric = model.evaluate(X_test, y_test)
```

```

print("Test Loss:", test_loss)
print("Test Metric:", test_metric)


# SECTION 8 – MODEL IMPROVEMENT
# QUESTION 7 (2 marks): Add another hidden layer (64 neurons).
# Keep the same activation function as used previously.
# Compare performance and explain the difference.
# ANSWER 7: Training took longer. More neurons mean more computation, more accurate but uses more resou

model_2 = Sequential([
    Dense(64, activation="relu", input_shape=(X_train.shape[1],)),
    Dense(32, activation="relu"),
    Dense(1)
])

model_2.compile(
    optimizer="adam",
    loss="mse"
)

history_2 = model_2.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=50,
    verbose=0
)

print("Model 2 trained.")

# SECTION 9 – ACTIVATION FUNCTION CHANGE
# QUESTION 8(1 mark): Replace the activation function with a different one.
# Observe training behaviour and explain the difference.
# ANSWER 8: Training is longer. relu goes from 0-1 and tanh is -1-1 so tanh is more complex as it cover

model_3 = Sequential([
```

```
    Dense(32, activation="tanh", input_shape=(X_train.shape[1],)),
    Dense(1)
])

model_3.compile(
    optimizer="adam",
    loss="mse"
)

history_3 = model_3.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=50,
    verbose=0
)

print("Model 3 trained.")

# Add your reflections as comments at each stage in the code.

```

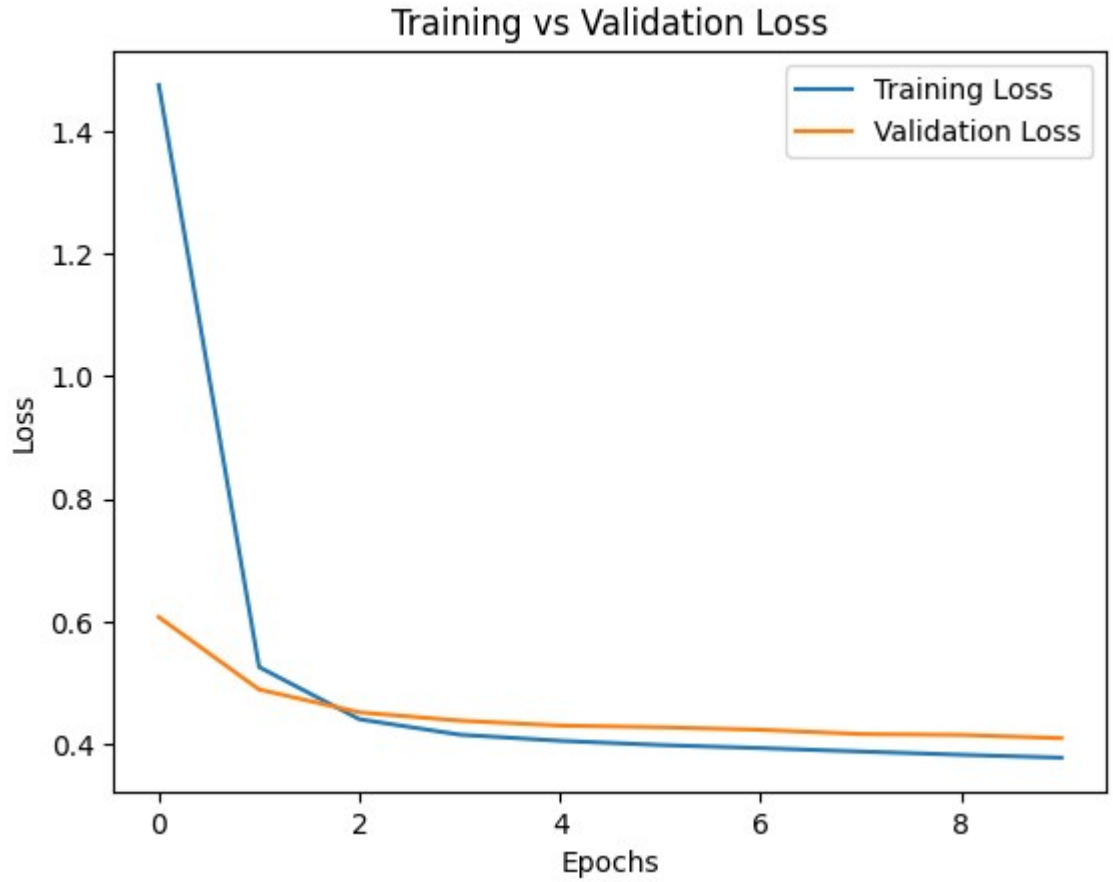
```
Feature shape: (20640, 8)
Target shape: (20640,)
Training samples: 16512
Test samples: 4128
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `inpu
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_11"

Layer (type)	Output Shape	Param #
dense_29 (Dense)	(None, 8)	72
dense_30 (Dense)	(None, 1)	9

Total params: 81 (324.00 B)
Trainable params: 81 (324.00 B)
Non-trainable params: 0 (0.00 B)

```
Training complete.
```




```
129/129 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.0013 - loss: 0.3838
Test Loss: 0.3889158368110657
Test Metric: 0.0019379844889044762
Model 2 trained.
Model 3 trained.
```